
```
if (solved) {
    displayBoard(board);
    for (let move of solutionMoves) {
        document.write("Move " + move + "<br />");
        makeMove(board, move);
        document.write("<br />"); // Print a newline.
        displayBoard(board);
        document.write("<br />"); // Print a newline.
    }
    document.write("Solved in " + solutionMoves.length + " moves:<br />");
    document.write(solutionMoves.join(", ") + "<br />");
    return true; // Puzzle was solved.
} else {
    return false; // Unable to solve in maxMoves moves.
}
}
```

If `attemptMove()` finds a solution, the program runs through all the moves gathered in the `solutionMoves` list or array and displays the board after each slide. This proves to the user that the moves collected by `attemptMove()` are the real solution to the puzzle. Finally, the `solve()` function itself returns `True`. If `attemptMove()` is unable to find a solution, the `solve()` function simply returns `False`.

The attemptMove() Function

Let's take a look at `attemptMove()`, the core recursive function behind our tile-solving algorithm. Remember the tree graph that a sliding-tile puzzle produces; calling `attemptMove()` for a certain direction is like traveling down that edge of this graph to the next node. A recursive `attemptMove()` call moves further down the tree. When this recursive `attemptMove()` call returns, it backtracks to a previous node. When `attemptMove()` has backtracked all the way to the root node, the program execution has returned to the `solve()` function.

The Python code for `attemptMove()` begins as follows:

Python

```
def attemptMove(board, movesMade, movesRemaining, prevMove):
    """A recursive function that attempts all possible moves on `board`
```